E-mail Phishing Detector

1. Abstract

Phishing attacks remain one of the most prevalent cybersecurity threats, targeting individuals and organizations through deceptive emails designed to steal sensitive information. This project presents an automated solution built with Python and Flask that detects phishing threats in real time. The system connects to a Gmail inbox via IMAP, scans each email for suspicious URLs using the VirusTotal API and keyword-based heuristics, quarantines flagged emails, sends alert notifications, and logs all activity in a SQLite database. A modern Flask web dashboard provides a clean interface for login, scanning, viewing, and deleting emails — all without manual intervention.

2. Objectives

- Automate the detection of phishing emails from a Gmail inbox using IMAP
- Extract and analyze URLs using the VirusTotal API for malicious content
- Apply keyword-based heuristic detection as a fast pre-filter
- Quarantine suspicious emails by applying Gmail labels
- Send real-time alert emails to notify the account owner of threats
- Log all flagged emails in a local SQLite database for audit purposes
- Provide a Flask-based web dashboard for interactive email management

3. System Architecture

The system follows a layered pipeline architecture where each component handles a distinct phase of the detection and response workflow:

Component	Role
`login.html / app.py`	User authenticates with Gmail + App Password via Flask
`email_handler.py`	Connects to Gmail over IMAP SSL and fetches all inbox emails
`phishing_detection.py`	Keyword scan first; then VirusTotal API check for URLs
`quarantine.py`	Applies 'Quarantine' Gmail label to flagged emails
`alert.py`	Sends SMTP alert email back to the logged-in account
`logger.py`	Persists flagged email details to SQLite (phishing_alerts.db)
`dashboard.html`	Displays scan results, stats, view modal, delete button

Tarun Chand | chandtarun1234@gmail.com

Detection Flow:

```
User Login (Gmail + App Password) | v Fetch Emails via IMAP --> Keyword Check -
-> (Fast: phishing? flag immediately) | v (if URL found) VirusTotal API Check /
\ Safe Email Phishing Detected | | Show as Safe Quarantine + Alert + Log
```

4. Implementation Details

app.py — Flask Application Core

The central Flask app manages 5 routes: **/ (login)**, **/dashboard**, **/scan**, **/delete/<id>**, and **/logout**. User credentials are stored in the Flask session (not in the DB). The /scan route is called asynchronously via AJAX — it fetches emails, runs detection, stores results in the session, and returns JSON. The dashboard reads from session['scan_results'] making it fast to reload.

```
app.secret_key = 'email-phishing-secret-key'

Routes: GET/POST / -> login(), stores email+password in session
GET /dashboard -> renders dashboard with session['scan_results']
POST /scan -> connects IMAP, scans, stores results, returns JSON
POST /delete/ -> permanently deletes email from Gmail GET /logout
-> session.clear() + redirect
```

email_handler.py — IMAP Email Fetching

Connects to Gmail using IMAP4_SSL on imap.gmail.com. Fetches ALL emails from the inbox (not just unread) using RFC 822 format. Extracts the sender email using parseaddr() from email.utils for clean parsing. Handles both multipart and plain text emails, decoding the body with error='ignore' for robustness.

phishing_detection.py — Dual-Layer Detection Engine

Uses two detection layers for efficiency and accuracy:

- Layer 1 — Keyword Heuristics: Checks for phishing trigger words (lottery, won, winner, urgent, verify, click, free money, confirm your details, clicking, claim, claiming). This is checked first as it requires no API call.
- Layer 2 — VirusTotal API: If a URL is found in the email body (via regex), it is Base64 URL-encoded and submitted to the VirusTotal v3 API. If malicious > 0 in last_analysis_stats, the email is flagged.

```
VT_API_KEY = 'YOUR_VIRUSTOTAL_API_KEY'

Priority Order (in app.py scan route):
if keyword_phishing_check(body): -> phishing = True (no API call)
elif contains_url(body): -> check_url_virustotal(body)
```

quarantine.py — Gmail Quarantine

Creates a 'Quarantine' label in Gmail and applies it to the flagged email using Gmail's IMAP extension (X-GM-LABELS). This isolates the email without deleting it, preserving it for further review.

alert.py — SMTP Alert System

Sends an alert email using SMTP with STARTTLS on port 587 (smtp.gmail.com). The alert is sent from and to the same account used for scanning — no separate admin email needed. Uses MIME Text for the message body.

logger.py — SQLite Logging

Persists all flagged email records to phishing_alerts.db using SQLite3. Creates the flagged_emails table if it doesn't exist (sender, subject, body). Every detected phishing email is stored permanently for audit and review.

5. Web Dashboard (Flask + HTML/CSS)

The project includes a fully functional Flask web dashboard accessible via browser, replacing the original command-line interface with a modern UI.

Feature	Description
Login Page	Enter Gmail address + App Password; dark theme with background image
Stats Cards	Shows Total Emails / Phishing Count / Safe Count at a glance
Scan Button	AJAX call to /scan; shows loading state without page reload
Email Table	Lists sender, View button, Delete button, and color-coded status
View Modal	Click View to see full email body, sender, and threat status in a popup
Delete Email	Permanently removes email from Gmail inbox via IMAP expunge
Logout	Clears session and redirects to login page
Footer	Displays author contact : chandtarun1234@gmail.com

6. Project Structure

```
Email-phishing-detector/

■   app.py # Flask app — all routes main.py # CLI runner
■   (original flow) email_handler.py # IMAP connection +
■   email fetching phishing_detection.py # Keyword check +
■   VirusTotal API quarantine.py # Gmail quarantine label
■   alert.py # SMTP alert sender logger.py # SQLite logging
■   phishing_alerts.db # SQLite database (auto-created)
■   requirements.txt # Python dependencies
■   templates/
■   login.html # Login page
■   dashboard.html # Main dashboard
■   static/
■   style.css # Dashboard styles
■   login.css # Login page styles
■   script.js # AJAX scan + modal + delete logic
■   bp.jpg # Login background image
```

7. Requirements & Setup

Requirement	Details
Python	3.6 or higher
Flask	pip install flask
Requests	pip install requests
Gmail Account	2-Step Verification enabled
Gmail App Password	Generated from Google Account Security settings
VirusTotal API Key	Free account at virustotal.com
IMAP	Enable IMAP in Gmail Settings > Forwarding and POP/IMAP

Setup Steps:

1. Clone or download the repository
2. Install dependencies: pip install flask requests
3. Enable IMAP in your Gmail settings
4. Generate a Gmail App Password (Google Account > Security > App Passwords)
5. Add your VirusTotal API key in phishing_detection.py
6. Run: python app.py
7. Open browser at: http://127.0.0.1:5000
8. Login with your Gmail address and App Password

Tarun Chand | chandtarun1234@gmail.com

8. Test Cases

Test Case	Input	Expected Output	Result
Safe Email	Email with no URLs, no keywords	Marked Safe, not logged	■ Pass
Keyword Phishing	Email body contains 'lottery winner'	logged immediately, quarantined + logged	■ Pass
URL Phishing	Email with known malicious URLVirusTotal flags URL	quarantine + alert + log	■ Pass
False Positive	Email with low-reputation URL	Flagged, logged for review	■ Review
No Emails	Empty inbox	Dashboard shows 0 results, no error	■ Pass
Wrong Password	Invalid Gmail credentials	Exception returned as JSON error	■ Pass

9. Error Handling

■ **Invalid Gmail Credentials:** IMAP login failure is caught by try/except in app.py /scan route and returned as a JSON error message to the frontend.

■ **VirusTotal Rate Limit:** The API call uses a 6-second timeout. If the rate limit is exceeded, the URL check returns False (treated as safe) to avoid blocking.

■ **Empty Inbox:** fetch_emails() returns an empty list; the dashboard shows 0 results gracefully without crashing.

■ **Network Failure:** All external calls are wrapped in try/except; errors are returned as JSON with HTTP 500 status.

■ **SMTP Alert Failure:** alert.py catches SMTP exceptions and prints the error without crashing the main scan flow.

10. Security Considerations

■ **No Hardcoded Credentials:** Gmail credentials are entered at runtime via the login form and stored only in the Flask session — never in source code.

■ **API Key Protection:** The VirusTotal API key is a placeholder in source code. In production, store it in an environment variable using os.environ.

■ **App Password Only:** The system uses Gmail App Passwords, not the main account password, limiting the blast radius of any credential leak.

■ **Session-Based Auth:** All routes check for session['email'] before executing — unauthorized access is redirected to login immediately.

■ **Email Body Sanitization:** Email bodies are displayed using Jinja2's auto-escaping (| e filter) in the dashboard template to prevent XSS.

11. Results & Conclusion

During testing, the system successfully detected and quarantined phishing emails containing malicious URLs as identified by VirusTotal. Keyword-based detection provided near-instant flagging for obvious phishing attempts without any API calls, improving performance significantly. Legitimate emails were processed without false positives in most cases. The Flask dashboard provided a clean and intuitive interface for managing inbox emails in real time.

Tarun Chand | chandtarun1234@gmail.com

The system demonstrates a practical, automated approach to email security that can be deployed by individuals or small teams to strengthen their cyber security posture with minimal setup. The dual-layer detection (keywords + VirusTotal), automated quarantine, and SMTP alert system together form a complete threat response pipeline that operates without any manual intervention.

12. Future Enhancements

- → Machine learning classifier trained on phishing email datasets for improved accuracy
- → Support for multiple email providers (Outlook, Yahoo) via configurable IMAP settings
- → Real-time email monitoring with scheduled background scanning (APScheduler)
- → Dashboard analytics with graphs showing phishing trends over time
- → Email header analysis (SPF, DKIM, DMARC) for domain spoofing detection
- → Attachment scanning for malicious file types (.exe, .js, macro-enabled docs)
- → Docker containerization for one-command deployment
- → Environment variable support (.env file) for secure API key management

▪▪▪ About the Author

Tarun Chand

▪ Graphic Era Hill University, Bhimtal, Uttarakhand ▪ chandtarun1234@gmail.com